# Guion paso a paso del video — Sprint 3 (prueba de lo que se hizo)

**Duración:** ≈ 8–9 minutos · **Herramienta:** el navegador (Swagger) y tu bandeja de correo · **Qué se prueba:** todo lo del Sprint 3 — editar el negocio, el servicio y el recurso; desactivar y reactivar un servicio; **cerrar un recurso**; **completar una reserva**; los **6 reportes**; el **correo** que llega al cliente cuando le cancelan; y que un **cliente eliminado** deja de mostrar sus datos.

Mismo estilo que el guion del Sprint 2. Cada paso dice **qué hacer, qué decir y qué HU/cambio estás probando**.

> **HU** = historia de usuario. **P5**/**P12** = las dos peticiones que Calidad pidió en la revisión de supuestos. Habla mientras haces cada paso y calla mientras escribes; no hace falta hablar de corrido.

## 1. Antes de grabar (10 minutos)

1. Abre `https://proyecto2026-2-5zoo.onrender.com/actuator/health` y recarga hasta ver `{"status":"UP"}` (si estaba dormida tarda hasta 2 min). Hazlo 5 minutos antes.
2. Abre **3 pestañas** de `https://proyecto2026-2-5zoo.onrender.com/swagger-ui.html`: **PROVEEDOR**, **CLIENTE**, **ADMIN**.
3. Ten a mano un **bloc de notas** para ir pegando cada `id` (negocio, servicio, recurso, reservas, cierre, cliente).
4. Elige una **fecha de lunes a viernes**, de la próxima semana, formato `AAAA-MM-DD` (en el guion: `<fecha>`). Úsala en todos los pasos.
5. El **cliente** debes crearlo con un **correo real tuyo** (ahí llegará el aviso de P5). Ten ese buzón abierto en otra pestaña.
6. Para los dos últimos pasos (reporte de plataforma y cliente eliminado) necesitas una **cuenta administradora con la app autenticadora en el celular**.
7. Contraseña de demo (8+ caracteres, con mayúscula, minúscula y símbolo); no la muestres en pantalla.
8. Pestañas para el inicio: GitHub Actions y SonarCloud.

**No hagas más de 2 tomas en 10 minutos** (el sistema limita a 5 registros por IP; esta demo usa 2: proveedor y cliente).

## 2. Recordatorio de Swagger

| Acción | Cómo |
|---|---|
| Probar una operación | Clic en la barra → **Try it out** → pega datos → **Execute**. Mira el **código** (200, 201, 409…) y el cuerpo |
| Iniciar sesión | `POST /auth/login` → copia `token` → arriba **Authorize** → pega solo el token. **Candado abierto en una operación = falta esto (saldrá 401)** |
| `PATCH` | Es «editar». Solo envías los campos que cambian |

## 3. El guion, escena por escena

> Las frases **en cursiva** son lo que dices; con tus palabras está bien, pero conserva las **negritas**.

### Escena 1 — Introducción (0:00–0:40)

**Qué haces:** muestra GitHub Actions (todo verde) y Sonar (Quality Gate OK).

**Qué dices:** *«Hola, soy [nombre], de Arquitectura. Este es el **Sprint 3** de la Plataforma de Reservas. Completamos las historias de **edición y mantenimiento** que faltaban, agregamos los **reportes** del negocio, y aplicamos lo que **Calidad** nos pidió: avisar al cliente por **correo** cuando le cancelan, y **ocultar los datos** de un cliente eliminado. En total son **833 pruebas automáticas**, **97 % de cobertura** y **48 endpoints**. Lo probamos todo desde Swagger.»*

### Escena 2 — Preparar un negocio (0:40–2:00) · pestaña PROVEEDOR y CLIENTE

Montamos rápido un negocio para poder editarlo y reservar. (Es lo del Sprint 2, en modo exprés.)

| # | Operación | Pega |
|---|---|---|
| 1 | PROVEEDOR: `POST /api/v1/providers` | `{"fullName":"Proveedor Demo","email":"demo.prov.<x>@gmail.com","cellphone":"30111122<xx>","password":"<contraseña> Demo"}` |
| 2 | PROVEEDOR: `POST /auth/login` → **Authorize** | `{"email":"<correo del proveedor>","password":"<contraseña>"}` |
| 3 | `POST /businesses/{businessId}/services` | `{"name":"Corte","durationMinutes":60,"priceCop":25000}` |

| | | |
|---|---|---|
| 4 | `POST /businesses/{businessId}/resources` | `{"name":"Sala 1","type":"SALA"}` |
| 5 | `PUT /services/{serviceId}/resources` | `{"resourceIds":["<resourceId>"]}` |
| 6 | `PUT /resources/{resourceId}/availability` | `{"days":[{"dayOfWeek":1,"ranges":[{"start":"08:00","end":"20:00"}]},{"dayOfWeek":2,"ranges"[{"start":"08:00","end":"20:00"}]},{"dayOfWeek":3,"ranges":[{"start":"08:00","end":"20:00"}{"dayOfWeek":4,"ranges":[{"start":"08:00","end":"20:00"}]},{"dayOfWeek":5,"ranges":[{"start":"08:00","end":"20:00"}]}]}` |
| 7 | CLIENTE: `POST /api/v1/users` (**tu correo real**) | `{"fullName":"Cliente Demo","email":"TU_CORREO_REAL@gmail.com","cellphone":"30111133<xx>","password":"<contraseña` |
| 8 | CLIENTE: `POST /auth/login` → **Authorize** | `{"email":"TU_CORREO_REAL@gmail.com","password":"<contraseña>"}` |
| 9 | CLIENTE: `POST /api/v1/bookings` | `{"serviceId":"<serviceId>","date":"<fecha>","startTime":"10:00","endTime":"11:00"}` |
| 10 | CLIENTE: `POST /api/v1/bookings` | igual con `"startTime":"14:00","endTime":"15:00"` |

**Qué dices:** *«Preparo un negocio con un **servicio** de 60 minutos a 25 mil pesos, una **sala** con horario de lunes a viernes, y un **cliente** con mi correo real, que hace **dos reservas**. Sobre esto probamos lo nuevo.»*

## Escena 3 — Editar el negocio (2:00–2:50) · HU-07 · pestaña PROVEEDOR

| # | Operación | Pega | Debe salir |
|---|---|---|---|
| 11 | `PATCH /api/v1/businesses/{businessId}` | `{"description":"Cortes y barba","address":"Calle 10 # 43-12","businessType":"BELLEZA","phone":"6045551234"}` | **200**, «La información del negocio fue actualizada» |
| 12 | CLIENTE: `GET /api/v1/businesses/{businessId}` | — | **200**: ahora trae dirección, tipo y teléfono |
| 13 | PROVEEDOR: el mismo `PATCH` con `{"phone":"12345"}` | | **400** «El teléfono no es válido» |

**Qué dices:** *«El proveedor **edita su negocio** (HU-07): le pone descripción, dirección, tipo y teléfono. El **cliente ya los ve** en el catálogo. Y si pongo un teléfono inválido, lo **rechaza**: el tipo y el teléfono son listas cerradas.»*

## Escena 4 — Editar servicio y recurso (2:50–3:40) · HU-10, HU-15 · pestaña PROVEEDOR

| # | Operación | Pega | Debe salir |
|---|---|---|---|
| 14 | `PATCH /api/v1/services/{serviceId}` | `{"priceCop":60000}` | **200**, precio 60000 |
| 15 | CLIENTE: `GET /api/v1/bookings/me` | — | La reserva A sigue en **25000** |
| 16 | PROVEEDOR: `PATCH /api/v1/resources/{resourceId}` | `{"name":"Sala Principal"}` | **200**, nombre nuevo |

**Qué dices:** *«Edito el **precio del servicio** a 60 mil (HU-10). Pero fíjense: la reserva que el cliente **ya hizo sigue en 25 mil**. Eso es clave: la reserva guarda una **copia** del precio, para que un cambio no altere lo ya acordado. Y edito el **nombre del recurso** (HU-15).»*

## Escena 5 — Cerrar un recurso (3:40–5:00) · HU-21 + P5 (correo) · pestaña PROVEEDOR

| # | Operación | Pega | Debe salir |
|---|---|---|---|
| 17 | `POST /resources/{resourceId}/closures` | `{"start":"<fecha> 12:00","end":"<fecha> 13:00","reason":"Mantenimiento"}` | **201**, `cancelledBookings:0` |
| 18 | CLIENTE: `GET /services/{serviceId}/availability?date=<fecha>` | — | 12:00 ya **no** aparece; 14:00 sí |
| 19 | CLIENTE: `POST /api/v1/bookings` con `"startTime":"12:00","endTime":"13:00"` | | **409** «fuera del horario» |

| | | | |
|---|---|---|---|
| 20 | PROVEEDOR: `POST /resources/{resourceId}/closures` | `{"start":"<fecha> 14:00","end":"<fecha> 15:00"}` | **409** `CONFIRMATION_REQUIRED`, 1 reserva afectada |
| 21 | El mismo con confirmación | `{"start":"<fecha> 14:00","end":"<fecha> 15:00","confirm":true}` | **201**, `cancelledBookings:1` (cancela la reserva B) |
| 22 | **Abre tu correo** | — | Llega «Tu reserva en Salón Demo fue cancelada» |

**Qué dices:** *«El proveedor puede **cerrar un recurso** un rato, sin cambiar su horario normal (HU-21). Cierro de 12 a 1: ese horario **desaparece** de la disponibilidad y el cliente **no puede reservar** ahí. Si cierro **encima de una reserva**, el sistema me **avisa y pide confirmación**; al confirmar, cancela esa reserva… y aquí está lo que pidió Calidad: al cliente le **llega un correo** avisándole. Esto es la petición **P5**.»*

> El correo es asíncrono: llega en segundos. Si no quieres esperar en cámara, córtalo en la edición.

## Escena 6 — Desactivar y reactivar el servicio (5:00–5:50) · HU-11, HU-12 · pestaña PROVEEDOR

| # | Operación | Pega | Debe salir |
|---|---|---|---|
| 23 | `POST /services/{serviceId}/deactivation` | `{"confirm":false}` | **409** `CONFIRMATION_REQUIRED`, 1 reserva afectada (la A) |
| 24 | El mismo con `{"confirm":true}` | | **200**, `active:false`, `cancelledBookings:1` (y otro correo al cliente) |
| 25 | CLIENTE: `POST /api/v1/bookings` (10:00–11:00) | | **404** «el servicio no está disponible» |
| 26 | PROVEEDOR: `POST /services/{serviceId}/reactivation` | (sin cuerpo) | **200**, `active:true` |

**Qué dices:** *«Mismo patrón para **desactivar un servicio** (HU-11): si tiene reservas, pide confirmación; al confirmar las cancela —y de nuevo **avisa por correo**— y el servicio **deja de poder reservarse**. Después lo **reactivo** (HU-12): es el mismo servicio, no uno nuevo.»*

## Escena 7 — Completar una reserva (5:50–6:20) · HU-27 · pestaña CLIENTE y PROVEEDOR

| # | Operación | Pega | Debe salir |
|---|---|---|---|
| 27 | CLIENTE: `POST /api/v1/bookings` (09:00–10:00) | `{"serviceId":"<serviceId>","date":"<fecha>","startTime":"09:00","endTime":"10:00"}` | **201** (reserva **C**) |
| 28 | PROVEEDOR: `POST /bookings/{C}/completion` | (sin cuerpo) | **409** «solo se puede completar cuando haya terminado su horario» |

**Qué dices:** *«El proveedor puede **marcar una reserva como completada** cuando el servicio ya pasó (HU-27). Como esta reserva es **futura**, el sistema no deja: exige que **ya haya terminado**. El caso exitoso, con una reserva pasada, está cubierto en las **pruebas automáticas**, porque en vivo tendríamos que esperar a que pase la hora.»*

## Escena 8 — Reportes (6:20–7:30) · HU-29 a 33 · pestaña PROVEEDOR

Pon en cada reporte `?from=<fecha>&to=<fecha>`.

| # | Operación | Debe salir |
|---|---|---|
| 29 | `GET /businesses/{businessId}/reports/bookings-by-status?from=<fecha>&to=<fecha>` | 1 confirmada (C), 2 canceladas (A, B) |
| 30 | `GET /businesses/{businessId}/reports/resource-occupancy?from=<fecha>&to=<fecha>` | la sala con minutos reservados y su % de ocupación |
| 31 | `GET /businesses/{businessId}/reports/top-services?from=<fecha>&to=<fecha>` | «Corte» con sus reservas |
| 32 | `GET /businesses/{businessId}/reports/booking-value?from=<fecha>&to=<fecha>` | valores en COP (según el precio guardado) |

**Qué dices:** *«El proveedor tiene **reportes** de su negocio: **reservas por estado** (una confirmada y dos canceladas), **ocupación** de cada recurso en porcentaje, **servicios más reservados** y el **valor** de las reservas. Todo esto es un **módulo nuevo de solo lectura**: no guarda nada, lo calcula la base de datos sobre lo que ya existe.»*

### Escena 9 — Administrador: resumen y cliente eliminado (7:30–8:40) · HU-34, HU-05 + P12 · pestaña ADMIN

| # | Operación | Pega / dónde | Debe salir |
|---|---|---|---|
| 33 | `POST /auth/login` **sin código** | `{"email":"<admin>","password":"<contraseña>"}` | **401** `MFA_REQUIRED` |
| 34 | el mismo **con** el código del celular → **Authorize** | `{"email":"<admin>","password":"<contraseña>","mfaCode":"<6 dígitos>"}` | **200** |
| 35 | `GET /api/v1/admin/reports/platform-summary?from=<fecha>&to=<fecha>` | **200**: negocios activos, reservas por estado, ocupación general | |
| 36 | `DELETE /api/v1/users/{id del cliente}`, campo **X-MFA-Code** con el código vigente | — | **204** |
| 37 | PROVEEDOR: `GET /businesses/{businessId}/bookings` | — | Las reservas del cliente ahora dicen «**Cliente eliminado**» y sin correo |

**Qué dices:** *«El **administrador** —que entra con **doble factor**— tiene un **resumen de toda la plataforma** (HU-34): negocios activos, reservas y ocupación general. Y aquí la segunda petición de Calidad, la **P12**: cuando el administrador **elimina una cuenta** (HU-05), sus reservas **dejan de mostrar su nombre y su correo** y quedan como "Cliente eliminado"; el resto del historial se conserva.»*

> El código de la app cambia cada 30 s. Si falla, espera al siguiente y no insistas: 5 fallos seguidos bloquean 15 minutos.

### Escena 10 — Cierre (8:40–9:00)

**Qué dices:** *«Con esto el proyecto queda **funcionalmente completo**: las 34 historias de usuario. En el Sprint 3 vimos **edición y mantenimiento**, **cierres**, **completar reservas**, los **reportes**, el **correo** al cliente y la **privacidad** del cliente eliminado. Gracias.»*

## 4. Después de grabar

Con el administrador y un código vigente, elimina el proveedor demo para dejar la base limpia: `DELETE /api/v1/users/{userId del proveedor}` (el `userId` salió en el paso 1). El cliente ya se eliminó en el paso 36.

## 5. Si algo sale mal

| Qué ves | Qué hacer |
|---|---|
| **401** | Falta el token en **Authorize**, venció (1 h) o es de otra cuenta. Haz login y pégalo de nuevo |
| **403** | Pestaña equivocada (token de cliente en algo de proveedor, o al revés) |
| **429** | Demasiados registros/intentos: espera 10–15 min |
| **400**/**409** al reservar | Fecha pasada o fin de semana, u hora fuera de 08:00–20:00 |
| El correo no llega | Revisa spam; en Brevo app.brevo.com/transactional/email/logs; el cliente debe tener un **correo real** (no `@example.com`) |
| Tarda la primera vez | Render estaba dormido; espera y recorta en la edición |

## 6. Para responder si preguntan (una línea)

- **Reportes sin tablas:** los datos ya existen; el módulo solo los lee y la base los suma. Si creciera, se usaría una vista materializada.
- **Correo tras confirmar y en otro hilo:** así un proveedor de correo lento o caído **no deshace ni demora** la cancelación; si el correo falla, se pierde pero la cancelación es válida.
- **¿Por qué Brevo y no el correo normal?** Render gratuito **bloquea SMTP**; Brevo se usa por su API HTTPS.
- **Cliente eliminado:** se borran su nombre y correo de las reservas, pero el historial (servicio, horario, precio) se conserva para los reportes.
- **Completar reserva en vivo:** necesita una reserva ya terminada; el caso exitoso está en las pruebas automáticas.